

ARCHITECTURE ET PROGRAMMATION D'ACCÉLÉRATEURS MATÉRIELS

Lecture 1 : Architecture and programming models

Julien Jaeger - julien.jaeger@cea.fr

1

Introduction

❖ Theme : *accelerators*

- ❖ State-of-the-art
- ❖ Accelerators architecture
- ❖ Accelerators programming
- ❖ Specific optimizations

❖ Prerequisites

- ❖ Basic notions in computer architecture
- ❖ Imperative programming (especially C)
- ❖ Notion in parallel programming

2

General outline

❖ Goal :

- ❖ Introduction to accelerators
- ❖ Expertise in CUDA programming

❖ Outline :

- ❖ Architecture
- ❖ Programming models
- ❖ Introduction to CUDA

❖ Teachers

- ❖ Julien Jaeger (julien.jaeger@cea.fr)
- ❖ Thibaut Pepin (thibaut.pepin@cea.fr)
- ❖ Cedric Chevalier (cedric.chevalier@cea.fr)

3

Lecture 1 Outline

❖ Overview of existing accelerators

❖ Nvidia GPU architectures

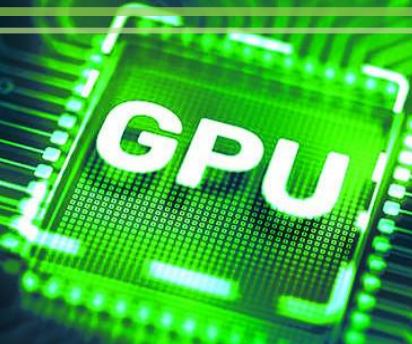
- ❖ NVIDIA Kepler
- ❖ NVIDIA Volta
- ❖ NVIDIA Ampere
- ❖ NVIDIA Grace+Hopper

❖ Overview of programming models for accelerators

❖ NVIDIA CUDA (part 1)

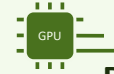
4

OVERVIEW OF ACCELERATORS



5

Accelerators



❖ Definition:

- ❖ A distinct resource (located on the same chip as the main processor or not) designed for different objectives than the central processor.

❖ In short:

- ❖ A processor dedicated to a special class of applications.
- ❖ Allows for better performance in its domain of expertise.



6

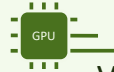
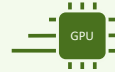
Accelerators – Interconnection



- ❖ Accelerators are attached to the processor(s) via an interconnect link.
- ❖ Behavior similar to a peripheral device:
 - ❖ The central processor controls access.
 - ❖ It also executes the main part of the code.
- ❖ Several types of interconnections:
 - ❖ Off-chip accelerators
 - ❖ On-chip accelerators

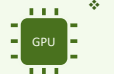
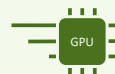


7



❖ Via the system bus:

- ❖ Nvidia GPGPU, Intel MIC
- ❖ Advantages:
 - ❖ Low cost
 - ❖ Easy integration
- ❖ Drawbacks:
 - ❖ Modest performance due to the generic nature of the bus
 - ❖ Suboptimal bandwidth and latency



Types of connections

❖ Via the processor bus:

- ❖ IBM Cell, AMD APU, ARM Grace + Nvidia Hopper
- ❖ Advantages:
 - ❖ Fast access
 - ❖ Possibility to share resources with the CPU
- ❖ Drawbacks:
 - ❖ Complex integration
 - ❖ Dependency on the processor



8

Accelerator Architecture

❖ Microarchitecture:

- ❖ Driven by the diversity of applications to be executed and the domain of expertise.
- ❖ Simplification compared to CPUs
 - ❖ (out-of-order execution, branch prediction, etc.).

❖ Exploited Parallelism:

- ❖ Generally fine-grained.
- ❖ Most exploit a form of Single Instruction Multiple Data (SIMD).

❖ But:

- ❖ Significant part of the work is left to the software stack and developers.

9

Accelerator Architecture

❖ Memory Hierarchy:

- ❖ Software-controlled memory (scratchpad memory).
- ❖ Hardware-managed cache (like a general-purpose CPU).
- ❖ Hardware interconnection between different compute processors.
- ❖ High bandwidth between the accelerator and global memory.

❖ Summary:

- ❖ Interesting possibilities for high-performance computing.
- ❖ But many aspects need to be managed manually.
- ❖ This directly influences how accelerators are programmed.

10

Accelerator Programming

❖ View of an accelerator as a worker peripheral.

- ❖ The main processor controls access to specialized computing resources:
 - ❖ Main processor (CPU): host
 - ❖ Accelerator: device
- ❖ General Concepts:
 - ❖ Data transfers
 - ❖ Execution of offloaded code in a different language
 - ❖ Manual execution management
 - ❖ Querying the accelerator for its status
 - ❖ Retrieving data back to the host

11

Overview of Accelerators

❖ Dedicated accelerators

- ❖ Cell (IBM)
- ❖ MIC or Xeon Phi (Intel)

❖ Graphics processors

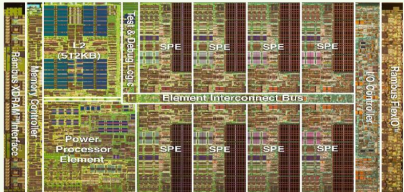
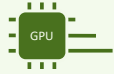
- ❖ NVIDIA
- ❖ AMD/ATI

❖ Reconfigurables architectures

- ❖ FPGA

12

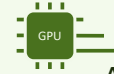
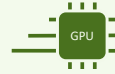
CELL processor (IBM)



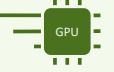
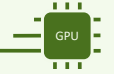
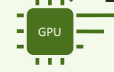
- ❖ Developed jointly by IBM, Sony, and Toshiba (2005+).
- ❖ 8 64-bit floating-point cores:
 - ❖ Called Synergistic Processor Elements (SPEs).
- ❖ One master 64-bit PowerPC processor capable of executing two threads.
- ❖ SPEs can process 128-bit operands, decomposed into four 32-bit words (SIMD).

13

MIC & Xeon Phi (Intel)

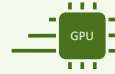
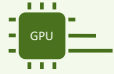


- ❖ Accelerator proposed by Intel:
 - ❖ Many-Integrated Core (MIC).
- ❖ Architecture:
 - ❖ x86 cores
 - ❖ Hyperthreading support
 - ❖ Cache coherence
- ❖ First version: KNF (2010)
- ❖ Latest version: KNL (2013) & KNM (2017)

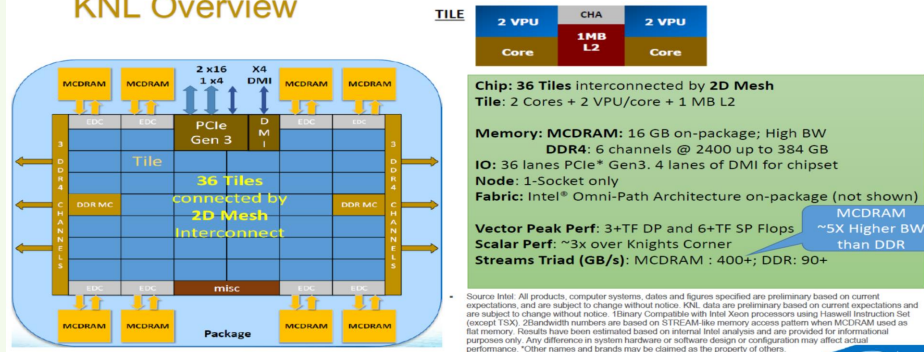


14

KNL

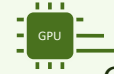


KNL Overview

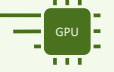
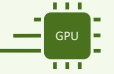
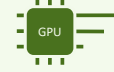


15

Graphics Processors



- ❖ Graphics cards initially for 2D and 3D display on screens:
 - ❖ Dedicated graphics processor(s).
- ❖ Impressive computing capabilities:
 - ❖ Optimized for image rendering (different objectives from general-purpose processors).
 - ❖ Ideal for processing regular data.
- ❖ Use for scientific computing:
 - ❖ GPGPU (General Purpose Graphics Processing Unit).



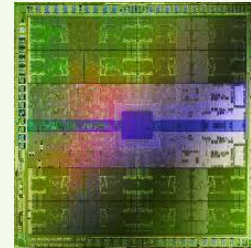
16

GPGPU

- ❖ Graphics cards on a system bus:
 - ❖ Interconnection on the motherboard.
 - ❖ Communications via the PCI-express port.
- ❖ Possibility to connect multiple graphics cards:
 - ❖ Multi-GPU programming.
 - ❖ Currently used in heterogeneous supercomputers.

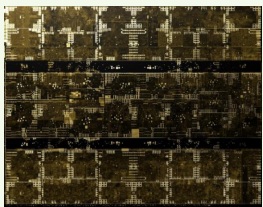
17

Nvidia Fermi Architecture (2010)



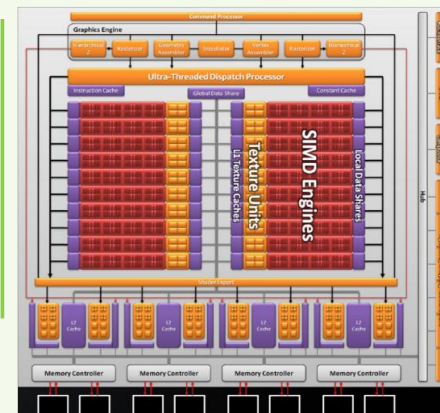
18

Nvidia Volta Architecture (2017)



19

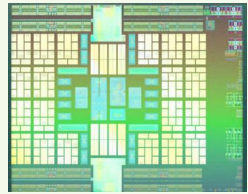
ATI Cypress Architecture (2009)



- ❖ Set of vector units:
 - ❖ SIMD Engines.
- ❖ Memory hierarchy:
 - ❖ L2 Cache.
- ❖ Peak performance:
 - ❖ 2.72 TFlops SP
 - ❖ 0.54 TFlops DP

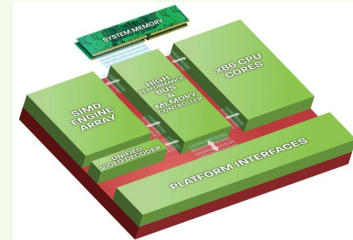
20

ATI Radeon Navi Architecture (2019)



21

AMD Fusion Llano Architecture (2011)

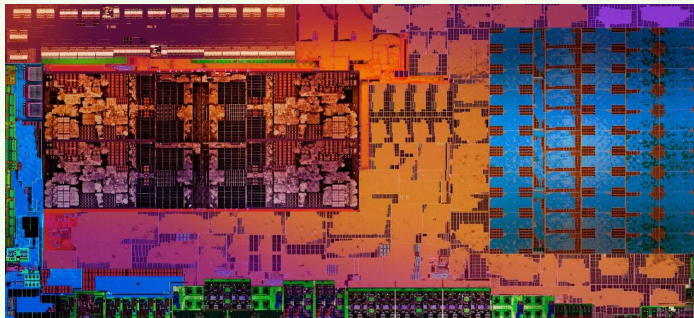


- ❖ Nom de code de l'architecture d'AMD/ATI : Fusion
- ❖ Design d'un APU (Accelerator Processing Unit)
- ❖ Sur la même puce :
 - ❖ CPU
 - ❖ Accélérateur de type GPU
- ❖ Première génération disponible en 2011
 - ❖ Llano avec technologie Bulldozer ou Bobcat
- ❖ <http://fusion.amd.com>

22

AMD APU Picasso Architecture

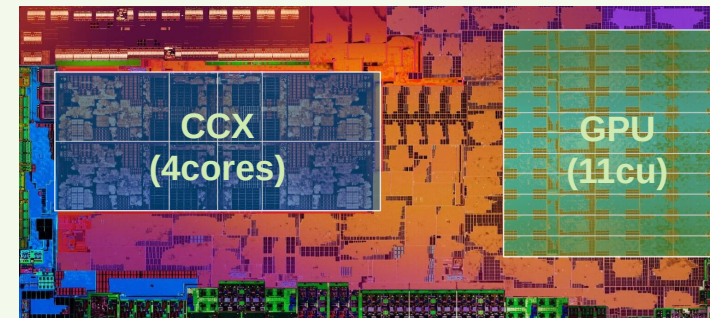
- Ryzen-bridge 2019



23

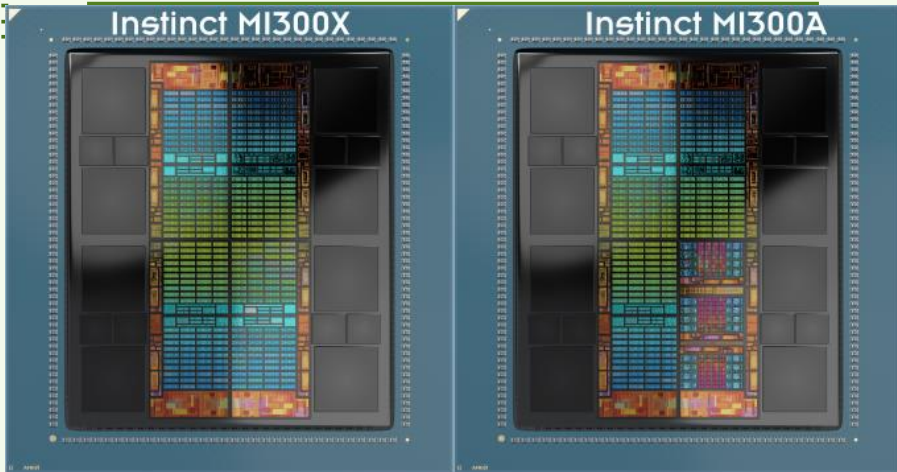
AMD APU Picasso Architecture

- Ryzen-bridge 2019



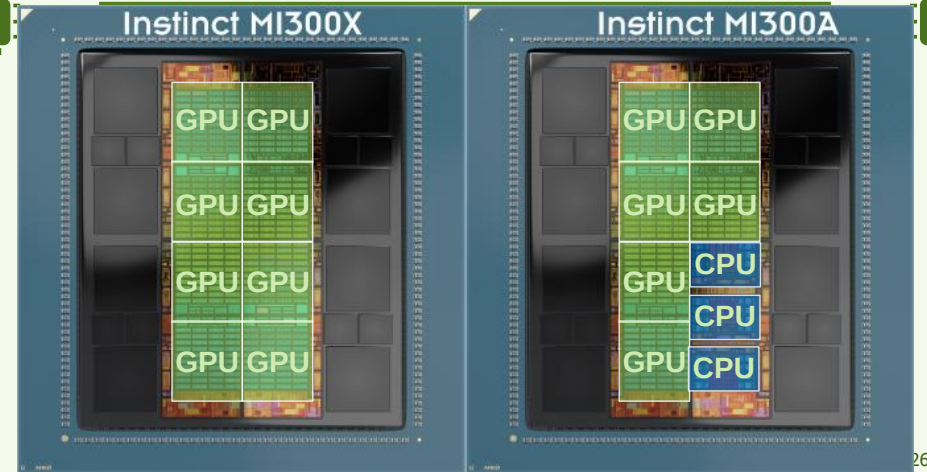
24

AMD MI300 – Frontier (2023)



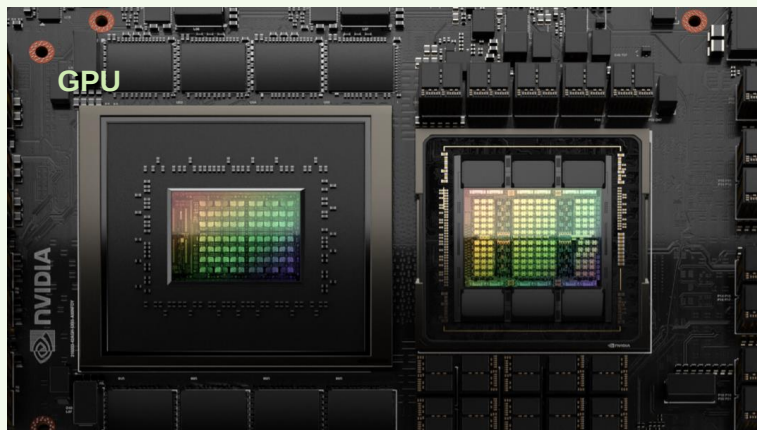
25

AMD MI300 – Frontier (2023)



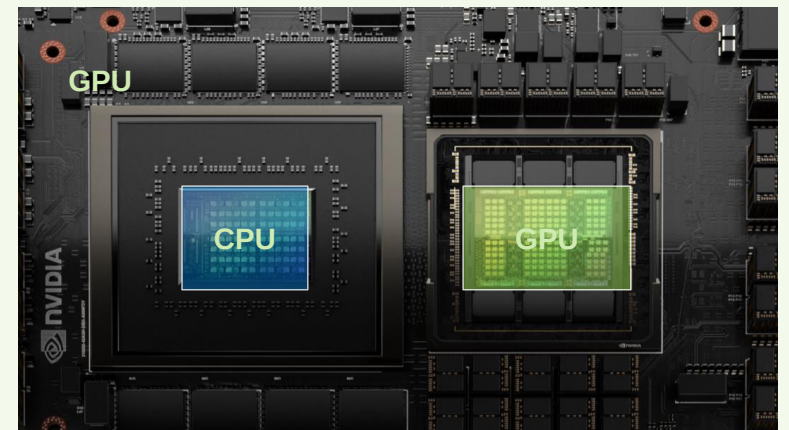
26

Nvidia Grace+Hopper (2022)



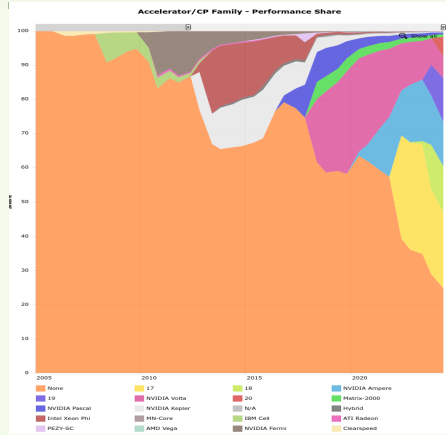
27

Nvidia Grace+Hopper (2022)



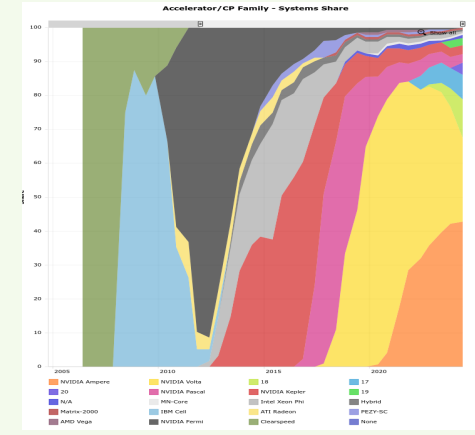
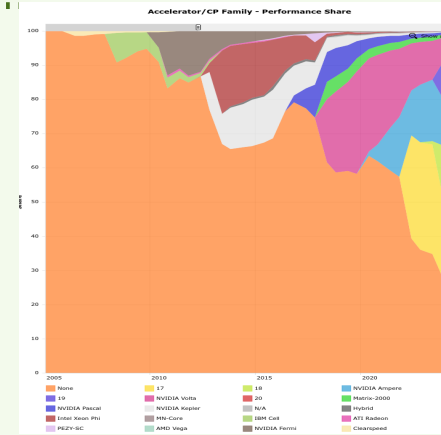
28

Evolution of accelerators in the Top500

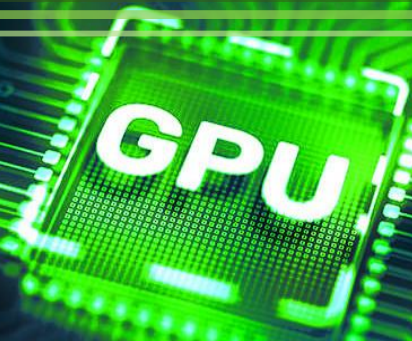


29

Evolution of accelerators in the Top500



NVIDIA GRAPHICS CARD ARCHITECTURE



31

NVIDIA GeForce Architecture

- ❖ Basic Building Block:
 - ❖ Very simple compute core.
 - ❖ Set of compute cores grouped in a stream multiprocessor.
- ❖ Why "stream"?
 - ❖ Designed to operate on a continuous data stream.
 - ❖ Synchronous language on each multicore.
- ❖ Design for Fine-Grained Parallelism

32

Nvidia Kepler Architecture (2012)

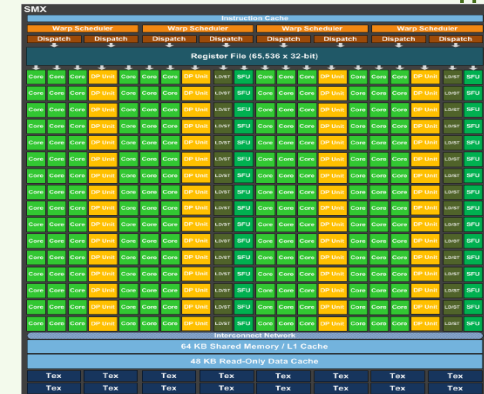
- ❖ 15 Streaming Multiprocessors (SMs).
- ❖ Arrangement perpendicular to the shared L2 cache.
- ❖ **Legend:**
 - ❖ Green: Compute core (double precision unit in orange).
 - ❖ Orange: Scheduler and dispatcher.
 - ❖ Light blue: Register bank and L1 cache.



33

Kepler : Streaming Multiprocessor

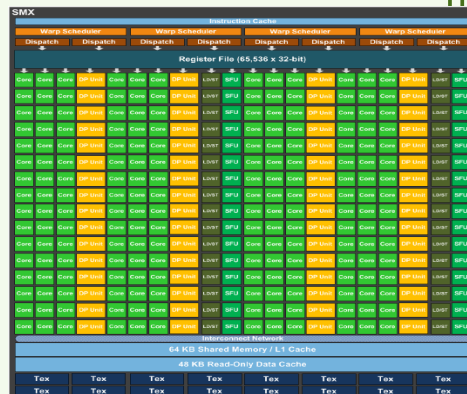
- ❖ Streaming Multiprocessor (SMX).
- ❖ 192 CUDA-compatible cores:
 - ❖ ALU unit.
 - ❖ Single-precision floating-point unit.
- ❖ 64 double-precision floating-point units.
- ❖ 32 special units (SFU).
- ❖ 32 load/store units.



34

Kepler : Streaming Multiprocessor

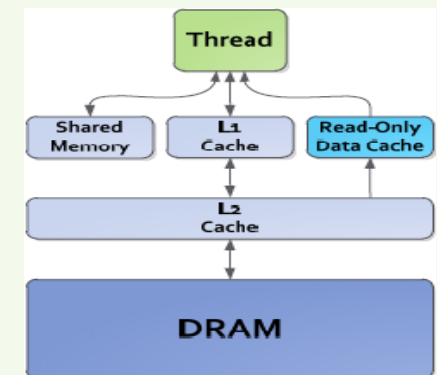
- ❖ SIMT execution type:
 - ❖ Single-Instruction Multiple Thread.
- ❖ Similar to SIMD.
- ❖ Synchronous execution of compute cores.
- ❖ Each SMX manages and executes compute threads in batches of 32:
 - ❖ Notion of warp.
 - ❖ 4 warp schedulers per SMX.



35

Kepler: Memory Hierarchy

- ❖ Thread scheduled on a compute core on an SMX.
- ❖ Memory access via load or store instruction.
- ❖ Two possibilities:
 - ❖ Access to shared memory: no cache.
 - ❖ Access to global device memory: passes through two levels of cache.
- ❖ Physically, the same memory is used for L1 cache and shared memory:
 - ❖ Appearance of a read-only cache (48 KB).



36

Nvidia Volta (2017)



- ❖ 6 GPCs (Graphics Processing Clusters).
- ❖ 7 TPCs (Texture Processing Clusters) per GPC.
- ❖ 2 SMs per TPC.
- ❖ Total: $2 * 7 * 6 = 84$ SMs!

37

Nvidia Volta (2017)



- ❖ 5376 single-precision floating-point units (14 Tflops) and same amount of integer units.
- ❖ 2688 double-precision floating-point units (7 Tflops).
- ❖ 672 tensor cores (112 Tflops).
- ❖ + 336 texture units.

38

Nvidia Ampere (2020)

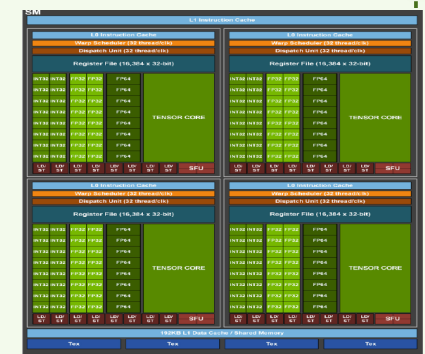


- ❖ 8 GPCs (Graphics Processing Clusters).
- ❖ 8 TPCs (Texture Processing Clusters) per GPC.
- ❖ 2 SMs per TPC.
- ❖ Total: $2 * 8 * 8 = 128$ SMs!

39

Nvidia Ampere: SM view

- ❖ 4 processing blocks per SM.
- ❖ 16 INT units: 8192 (21 Tops).
- ❖ 16 single-precision floating-point units: 8192 (21 Tflops).
- ❖ 8 double-precision floating-point units: 4096 (10 Tflops).
- ❖ 1 tensor core (112 Tflops).



40

Nvidia Hopper (2023)



- ❖ 8 GPCs (Graphics Processing Clusters).
- ❖ 9 TPCs (Texture Processing Clusters) per GPC.
- ❖ 2 SMs per TPC.
- ❖ Total: $2 * 9 * 8 = 144$ SMs!

41

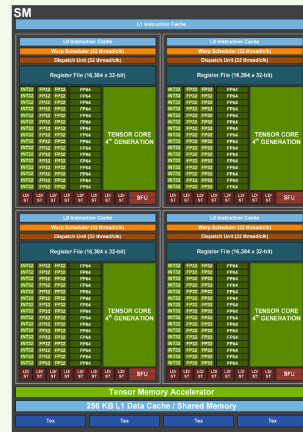
Hopper : SM view

- ❖ 4 × Processing Blocks:
 - ❖ 16 INT32 units.
 - ❖ 32 single-precision units.
 - ❖ 16 double-precision units.
 - ❖ 8 load/store units.
 - ❖ 1 warp scheduler.
 - ❖ 1 dispatch unit.
- ❖ Maximum 64 active warps.



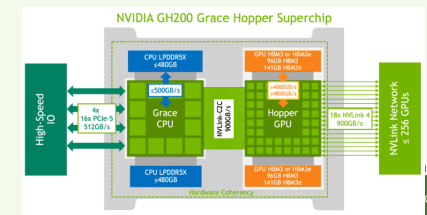
Hopper Performance

- ❖ "Classical" computing:
 - ❖ FP32: 60 Tflops.
 - ❖ FP64: 30 Tflops.
- ❖ "AI" computing (tensor cores):
 - ❖ FP64: 60 Tflops.
 - ❖ TF32: 1 Pflops.
 - ❖ FP16: 2 Pflops.
 - ❖ FP8: 4 Pflops.
- ❖ Difference between FP32 and TF32:
 - ❖ FP32: 8-bit exponent/23-bit mantissa.
 - ❖ TF32: 8-bit exponent/10-bit mantissa.
 - ❖ FP16: 5-bit exponent/10-bit mantissa.



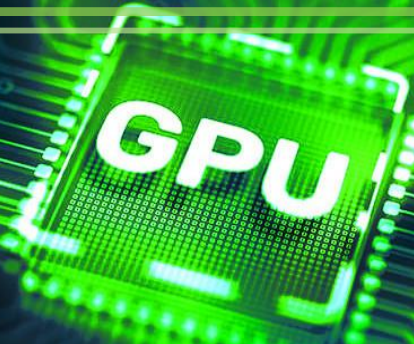
Grace-Hopper Superchip: The Future?

- ❖ "Grace" processor:
 - ❖ ARM Neoverse V2 architecture.
 - ❖ 72 cores.
- ❖ CPU and GPU on the same die to accelerate data transfers:
 - ❖ 2x 450GB/s.
 - ❖ Native unified memory guaranteed by a memory coherence system (C2C).
 - ❖ Beware of NUMA effects!



44

MODÈLES DE PROGRAMMATION POUR ACCÉLÉRATEURS



45

Programming Models

- ❖ Dedicated Languages:
 - ❖ Close To Metal (CTM), HIP/ROCm, CUDA.
- ❖ Portable Languages:
 - ❖ Low-level languages
 - ❖ OpenCL.
 - ❖ Directives based:
 - ❖ OpenMP, OpenACC.
 - ❖ High-Level Languages / DSLs:
 - ❖ Kokkos, Legion, RAJA, SYCL

46

ATI/AMD Programming

- ❖ Close to Metal (CTM) / CAL/IL:
 - ❖ Very low level.
 - ❖ Rarely used now.
- ❖ HIP/ROCm:
 - ❖ Provides a toolkit for writing GPU code and translating CUDA code to HIP.
- ❖ OpenCL/SYCL:
 - ❖ Supports OpenCL and SYCL standards.
 - ❖ AMD/ATI is part of the consortium.

47

NVIDIA Programming

- ❖ CUDA Language:
 - ❖ To be continued...

48

Directives de parallélisation

Proposal for directive-based programming:

- ❖ Adding #pragma (C, C++) in sequential code.
- ❖ Possibility to add these directives in already parallel code (MPI, OpenMP, etc.): depends on the model.

Proposed solutions:

- ❖ OpenACC
- ❖ OpenMP 4+

Advantages:

- ❖ Portability depending on the accelerators.
- ❖ Original (or even sequential) semantics can be preserved.

Drawbacks:

- ❖ Often need to finish "by hand".
- ❖ Sometimes limited scope.

49

NVIDIA CUDA

GPU

50

According to you...

❖ What do you think we will find in a GPU program?

- ❖ Which parts?
- ❖ Which steps?

51

Overview

❖ Two parts: host program and compute kernels.

❖ Need to transfer data between hosts and devices.

❖ Dedicated compilation chain due to compute kernels (different language).

❖ Link with a runtime (user space) and a driver (kernel space).

52

Programming Scheme

1. Initialization on host:
 - ❖ Memory allocation.
 - ❖ Reading inputs.
 - ❖ ...
2. Memory allocation on the GPU:
 - ❖ Manual management!
3. Transfer host → GPU.
4. Execution of a compute kernel:
 - ❖ Possibility to launch multiple kernels.
5. Transfer GPU → host.
6. Free GPU memory.
7. Free CPU memory.

53

Comment compiler ?

- ❖ Code Structure:
 - ❖ Part for the host.
 - ❖ Part for the device.
- ❖ In practice, these codes can be mixed in the same file.
- ❖ Host Programming:
 - ❖ C or C++.
- ❖ Device Programming:
 - ❖ CUDA (superset of C99 standard).
- ❖ Compilation Chain: NVCC:
 - ❖ As a programmer: use the nvcc compiler.
 - ❖ Do not hesitate to check the documentation provided with the SDK.

54

Example Program

- ❖ Example: Adding 2 vectors.
- ❖ Basic operation:
 - ❖ $c[i] = a[i] + b[i]$.
- ❖ How to program this compute kernel sequentially?
 - ❖ Simple loop.
 - ❖ Apply the basic operation to each element.

```
void vectAdd(
    double * a, double * b,
    double * c, int N) {
    int i ;
    for (i=0;i<N;i++) {
        c[i] = a[i] + b[i]
    }
}
```

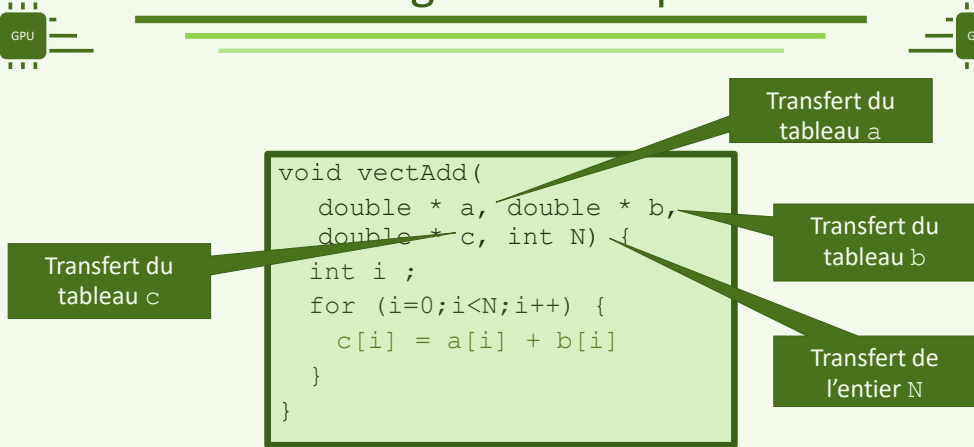
55

Porting Our Example

```
void vectAdd(
    double * a, double * b,
    double * c, int N) {
    int i ;
    for (i=0;i<N;i++) {
        c[i] = a[i] + b[i]
    }
}
```

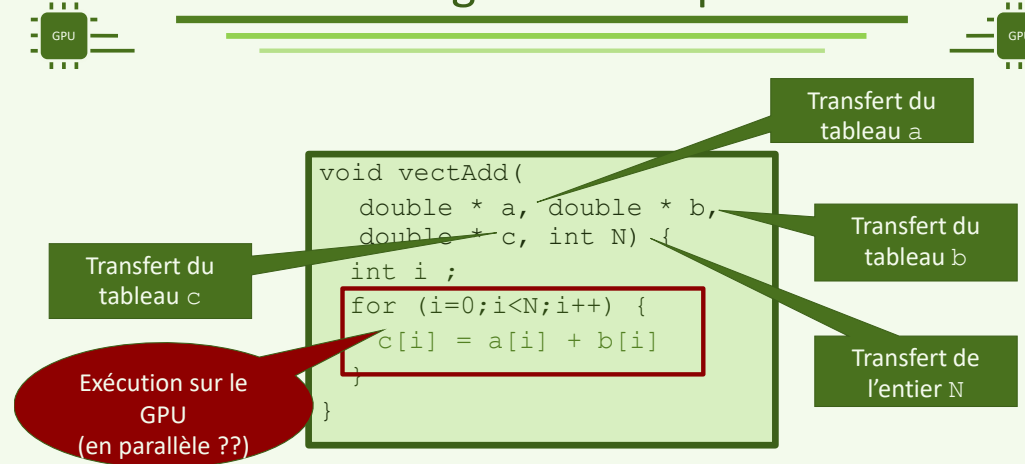
56

Porting Our Example



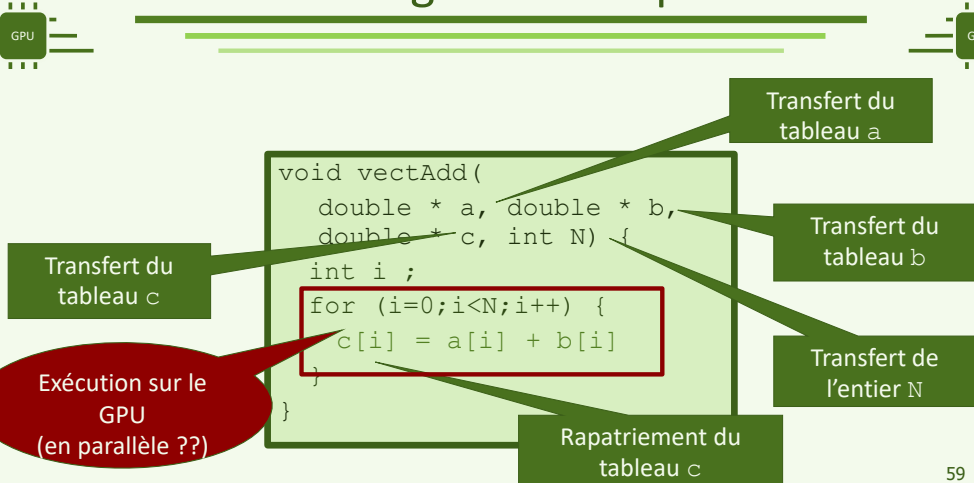
57

Porting Our Example



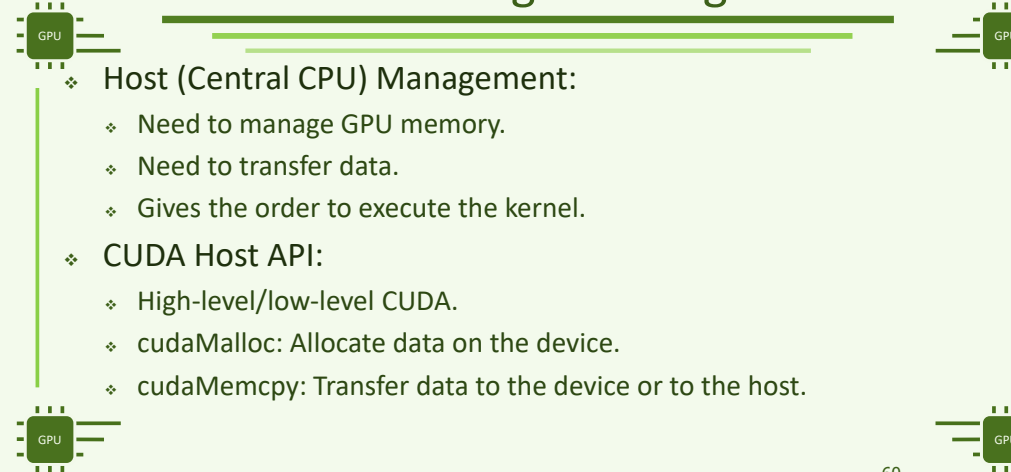
58

Porting Our Example



59

Host Programming



60

Host program

```
void vectAdd(
    double * a, double * b, double * c,
    int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a,
N*sizeof(double));
    cudaMalloc((void **)&d_b,
N*sizeof(double));
    cudaMalloc((void **)&d_c,
N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
cudaMemcpyHostToDevice);

    vectAddKernel(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

61

Host program

```
void vectAdd(
    double * a, double * b, double * c,
    int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMemcpy(d_a, a, N*sizeof(double),
cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
cudaMemcpyHostToDevice);

    vectAddKernel(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

Device
pointers

62

Host program

```
void vectAdd(
    double * a, double * b, double * c,
    int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMemcpy(d_a, a, N*sizeof(double),
cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
cudaMemcpyHostToDevice);

    vectAddKernel(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

Memory
allocation on
the device

63

Host program

```
void vectAdd(
    double * a, double * b, double * c,
    int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMemcpy(d_a, a, N*sizeof(double),
cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
cudaMemcpyHostToDevice);

    vectAddKernel(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

Data transfers
from host to
device

64

Host program

```
void vectAdd(
    double * a, double * b, double * c,
    int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a,
        N*sizeof(double));
    cudaMalloc((void **)&d_b,
        N*sizeof(double));
    cudaMalloc((void **)&d_c,
        N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

Kernel invocation
(will execute on
the device)

65

Host program

```
void vectAdd(
    double * a, double * b, double * c,
    int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

Data transfer
from device to
host

66

Host program

```
void vectAdd(
    double * a, double * b, double * c,
    int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a,
        N*sizeof(double));
    cudaMalloc((void **)&d_b,
        N*sizeof(double));
    cudaMalloc((void **)&d_c,
        N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

Free memory
allocated on
device

67

Execution model

- Now remains to execute the kernel on the device
- How to program the following code on the *device* ?

```
for (i=0; i<N; i++)
    c[i]=a[i]+b[i];
```

68

Execution model

- Now remains to execute the kernel on the device
- How to program the following code on the *device* ?


```
for (i=0;i<N;i++)
    c[i]=a[i]+b[i];
```
- The CUDA model is a model based on threads
 - At invocation of a kernel, launches a bunch of threads
 - Each thread executes the *kernel*
 - Quite similar to passing the kernel function to several pthread_create invocations
- CUDA programming is *multithreaded*
 - Need to think per thread, and not per loop or loop iteration

69

Execution model

- Now remains to execute the kernel on the device
- How to program the following code on the *device* ?


```
for (i=0;i<N;i++)
    c[i]=a[i]+b[i];
```
- The CUDA model is a model based on threads
 - At invocation of a kernel, launches a bunch of threads
 - Each thread executes the *kernel*
 - Quite similar to passing the kernel function to several pthread_create invocations
- CUDA programming is *multithreaded*
 - Need to think per thread, and not per loop or loop iteration
- Code du *kernel* dans notre cas :

```
int i = index; /* Update i with thread index */
c[i] = a[i] + b[i];
```

70

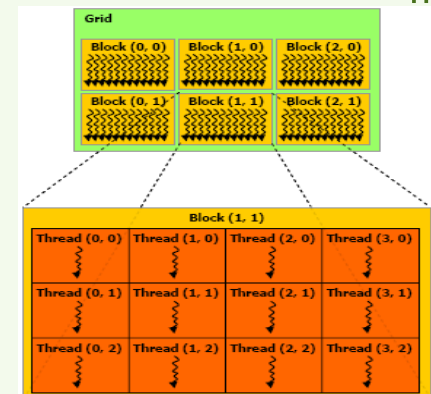
Threads hierarchy

- How to Know the Identifier of a Thread?
 - This is related to the organization of threads within the CUDA execution model.
- CUDA defines the concepts of blocks and grids to hierarchically organize threads:
 - A grid contains blocks.
 - A block contains threads.
- Both grids and blocks can have 1 to 3 dimensions (logical representation, not physical).
- In Practice ?

71

Threads hierarchy

- Example with
 - A grid with blocks organized in 2D
 - 6 blocks with threads also in 2D
 - 6 blocks in the grid with 12 threads per block
 - ➔ 72 threads in total
 - How to find a thread index ?
 - 2D array linearization

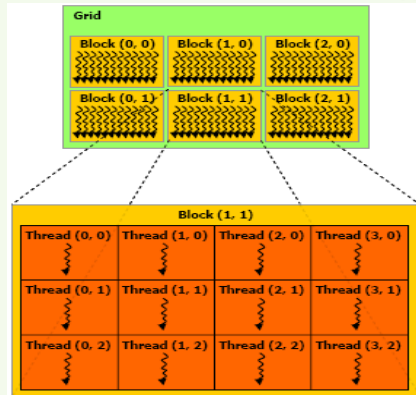


72

Threads hierarchy

CUDA defines several variables in the kernel

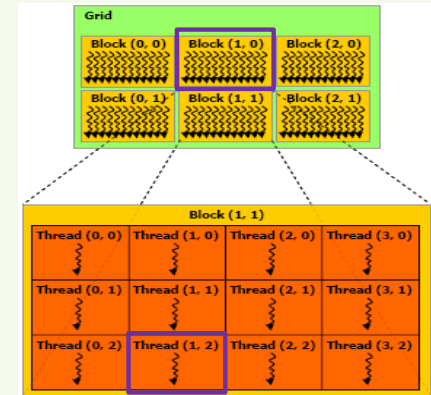
- `gridDim`: Contains the number of blocks in each dim
- `blockIdx`: Contains the coordinates of the block in the grid for the three dimensions
- `blockDim`: Contains the number of threads in each dim
- `threadIdx`: Contains the coordinates of the thread in its block for the three dimensions
- The type of these variables is a CUDA specific type:
 - `dim3`
 - It has three fields: `{x, y, z}` for values on each dimension



73

Threads hierarchy

Let's compute the index of the chosen thread

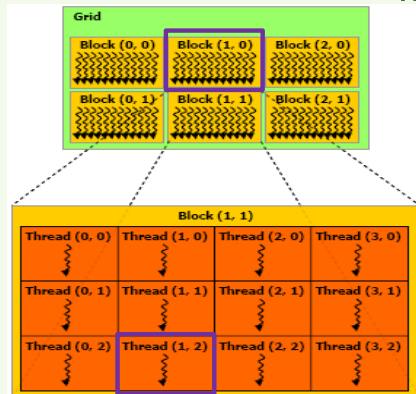


74

Threads hierarchy

Let's compute the index of the chosen thread

- `BlockIdx.x = 1; BlockIdx.y = 0`
- `GridDim.x = 3; GridDim.y = 2`

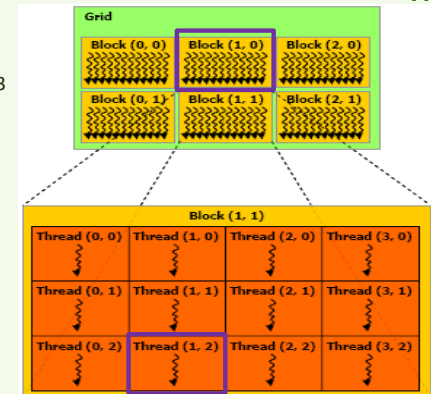


75

Threads hierarchy

Let's compute the index of the chosen thread

- `BlockIdx.x = 1; BlockIdx.y = 0`
- `GridDim.x = 3; GridDim.y = 2`
- $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$

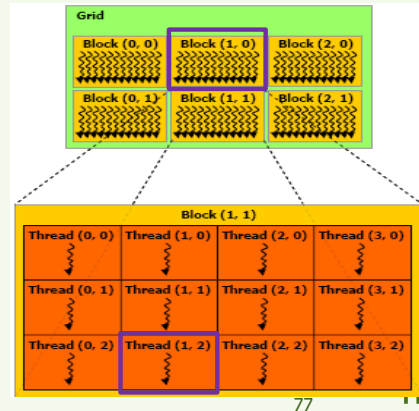


76

Threads hierarchy

Let's compute the index of the chosen thread

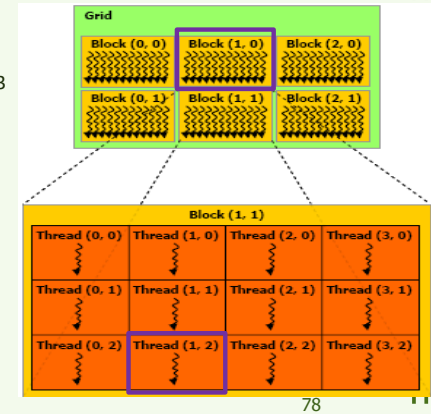
- BlockIdx.x = 1 ; BlockIdx.y = 0
- GridDim.x = 3 ; GridDim.y = 2
- Block_glob_id = BlockIdx.y x GridDim.x + BlockIdx.x = 0 x 3 + 1 = 1
- BlockDim.x = 4
- BlockDim.y = 3



Threads hierarchy

Let's compute the index of the chosen thread

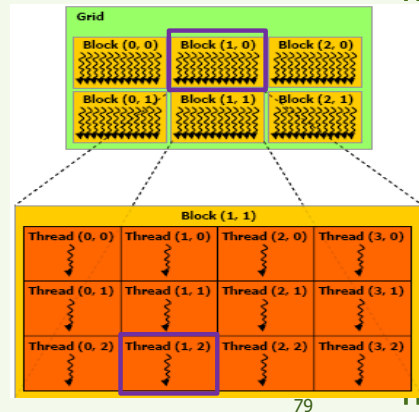
- BlockIdx.x = 1 ; BlockIdx.y = 0
- GridDim.x = 3 ; GridDim.y = 2
- Block_glob_id = BlockIdx.y x GridDim.x + BlockIdx.x = 0 x 3 + 1 = 1
- BlockDim.x = 4
- BlockDim.y = 3
- Nb_threads_per_block = BlockDim.x x BlockDim.y = 4 x 3 = 12



Threads hierarchy

Let's compute the index of the chosen thread

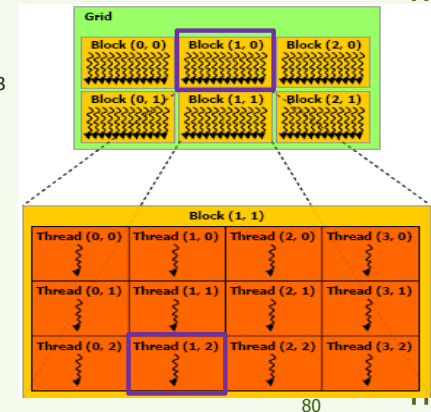
- BlockIdx.x = 1 ; BlockIdx.y = 0
- GridDim.x = 3 ; GridDim.y = 2
- Block_glob_id = BlockIdx.y x GridDim.x + BlockIdx.x = 0 x 3 + 1 = 1
- BlockDim.x = 4
- BlockDim.y = 3
- Nb_threads_per_block = BlockDim.x x BlockDim.y = 4 x 3 = 12
- threadIdx.x = 1
- threadIdx.y = 2
- BlockDim.x = 4



Threads hierarchy

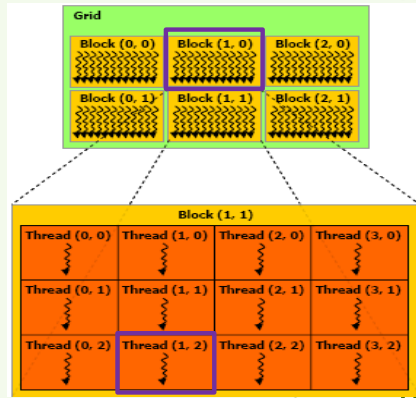
Let's compute the index of the chosen thread

- BlockIdx.x = 1 ; BlockIdx.y = 0
- GridDim.x = 3 ; GridDim.y = 2
- Block_glob_id = BlockIdx.y x GridDim.x + BlockIdx.x = 0 x 3 + 1 = 1
- BlockDim.x = 4
- BlockDim.y = 3
- Nb_threads_per_block = BlockDim.x x BlockDim.y = 4 x 3 = 12
- threadIdx.x = 1
- threadIdx.y = 2
- BlockDim.x = 4
- Thread_loc_id = threadIdx.y x BlockDim.x + threadIdx.x = 2 x 4 + 1 = 9



Threads hierarchy

- Let's compute the index of the chosen thread
 - BlockIdx.x = 1 ; BlockIdx.y = 0
 - GridDim.x = 3 ; GridDim.y = 2
- Block_glob_id = BlockIdx.y x GridDim.x + BlockIdx.x = 0 x 3 + 1 = 1
- BlockDim.x = 4
- BlockDim.y = 3
- Nb_threads_per_block = BlockDim.x x BlockDim.y = 4 x 3 = 12
- threadIdx.x = 1
- threadIdx.y = 2
- BlockDim.x = 4
- Thread_loc_id = threadIdx.y x BlockDim.x + threadIdx.x = 2 x 4 + 1 = 9
- Thread_glob_id = Block_glob_id x Nb_threads_per_block + Thread_loc_id = 1 x 12 + 9 = 21



81

CUDA programming

- Kernel declaration
 - Function attribute : `__global__`
- Compute thread index depending on the grid and blocks
 - Example on the right with a 1D grid and 1D blocks
- We add a test, checking the index is not greater than the array size
 - Correspond to the boundary of the original loop

```
__global__ void vecAddKernel(
    double *a,
    double *b,
    double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x
    + threadIdx.x ;

    if ( i < N ) {
        c[i] = a[i] + b[i];
    }
}
```

82

Kernels invocation

- How to choose the sizes of the grid and the blocks in the kernel ?
- The user specifies the sizes at kernel invocation
 - Several ways to do it
- Most common syntax of kernel invocation on the *device*

```
my_kernel<<<Dg, Db>>>>(arg1, arg2, arg3 );
```

 - Dg : grid size (e.i., number of blocks in each dimension) (type dim3)
 - Db : block size (e.i., number of threads in each dimension) (type dim3)
- Total number of blocks ?
- Thread number per block ?
- Total number of threads ?

83

Kernels invocation

- Blackwell introduced a new level
 - Thread block cluster
 - Between GRID and BLOCK
 - Not possible to integrate that into `<<<Dg, Db>>>`
 - Compatibility issue
 - compile-time kernel attribute
 - `__cluster_dims__(X,Y,Z)`
 - or use `cudaLaunchKernelEx`

84

Complete CUDA programm

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
               cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
               cudaMemcpyHostToDevice);
    cudaMemcpy(d_c, c, N*sizeof(double),
               cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
               cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i < N ) {
        c[i] = a[i]+b[i];
    }
}
```

85

Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
               cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
               cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
               cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i < N ) {
        c[i] = a[i]+b[i];
    }
}
```

86

Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
               cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
               cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
               cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i < N ) {
        c[i] = a[i]+b[i];
    }
}
```

87

Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
               cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
               cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
               cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i < N ) {
        c[i] = a[i]+b[i];
    }
}
```

88

Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

DEVICE

```
__global__ void vecAddKernel( double
    *a, __double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i < N ) {
        c[i] = a[i]+b[i];
    }
}
```

89

Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

DEVICE

```
__global__ void vecAddKernel( double
    *a, __double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i < N ) {
        c[i] = a[i]+b[i];
    }
}
```

90

Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

DEVICE

```
__global__ void vecAddKernel( double
    *a, __double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i < N ) {
        c[i] = a[i]+b[i];
    }
}
```

91

Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

DEVICE

```
__global__ void vecAddKernel( double
    *a, __double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

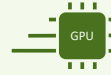
    if ( i < N ) {
        c[i] = a[i]+b[i];
    }
}
```

92

Programme CUDA complet

HOST

```
...  
void vectAdd(  
    double * a, double * b, double * c, int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a, N*sizeof(double));  
    cudaMalloc((void **)&d_b, N*sizeof(double));  
    cudaMalloc((void **)&d_c, N*sizeof(double));  
  
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
  
    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);  
  
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);  
}
```



DEVICE

```
__global__ void vecAddKernel( double  
    *a, double *b, double *c, int N) {  
  
    int i ;  
  
    i = blockIdx.x * blockDim.x +  
        threadIdx.x ;  
  
    if ( i < N ) {  
        c[i] = a[i]+b[i];  
    }  
}
```